

ADR-006 — Pipeline CI/CD GitHub Actions

Champ	Valeur
Statut	Accepté
Date	2026-03-08
Auteur	Pascal Bourgoin

Contexte

Le projet est déployé sur une VM Linux via Docker Compose. Sans pipeline automatisé, chaque déploiement est manuel et risque d'oublier des étapes (pull, écriture du `.env`, restart des conteneurs, vérification de santé). Les options évaluées :

1. **Script shell `deploy.sh` seul** — doit être lancé manuellement sur la VM, pas de validation préalable du code.
2. **GitHub Actions CI + CD séparés** — CI valide le code, CD déploie uniquement si CI réussit (retenu).
3. **Self-hosted runner sur la VM** — complexité d'installation et de maintenance du runner.

Décision

Deux workflows GitHub Actions hébergés sur des runners `ubuntu-latest` gratuits :

CI (`.github/workflows/ci.yml`) :

- Déclenché sur chaque push `main/develop` et PR vers `main`
- Job `build-java` : `./mvnw compile -B -q + ./mvnw package -DskipTests -B -q`
- Job `build-frontend` : `npm install + npm run build` (en parallèle de `build-java`)
- Aucun `docker build` en CI : les Dockerfiles relancent Maven en interne, ce serait un doublon

CD (`.github/workflows/cd.yml`) :

- Déclenché uniquement quand le CI **réussit** sur `main` (`workflow_run: completed`) ou manuellement (`workflow_dispatch`)
- Se connecte à la VM par SSH (`appleboy/ssh-action`)
- Transmet les secrets via `envs` (pas de heredoc fragile)
- Script SSH : `git pull` → écriture `.env` avec `printf` → création réseau Docker si absent → `docker compose down + up --build` → healthchecks

Les secrets de déploiement (`VM_HOST`, `VM_USER`, `VM_SSH_KEY`, `DB_ROOT_PASSWORD`, `DB_USER`, `DB_PASSWORD`) sont stockés dans GitHub Secrets et jamais versionnés.

Conséquences

Positives

- **Déploiement automatique** : chaque push sur `main` qui passe le CI déclenche le déploiement sans intervention manuelle.
- **Déploiement manuel** : le `workflow_dispatch` permet de redéployer à tout moment depuis l'interface GitHub sans accès SSH direct.
- **Pas de runner à maintenir** : les runners ubuntu-latest GitHub sont gratuits et toujours à jour.
- **Secrets sécurisés** : les credentials ne passent jamais dans les logs ni dans le code.
- **Rollback simple** : `git revert` + push sur `main` redéclenche le pipeline sur le commit précédent.

Négatives / Limites

- **Pas de tests d'intégration en CI** : les tests k6 (smoke, load, stress) nécessitent MySQL, Redis et MailHog. Ils sont lancés manuellement sur la VM après déploiement.
- **Déploiement sans downtime non garanti** : `docker compose down + up --build` crée une fenêtre d'indisponibilité (~15-30 s). Acceptable en contexte académique.
- **Clé SSH à gérer** : si la VM est recréée, il faut régénérer la clé et mettre à jour le secret GitHub.